Batch 9 — Admin Dashboard Website Leads Module

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Tue, 16 Jun at 08:12

Batch 9 — Admin Dashboard Website Leads Module

Meat Lovers CIMS powered by YohPal

9A.

database/migrations/022_add_website_lead_status_indexes.sql

```
CREATE INDEX idx_website_leads_status ON website_leads(lead_status);
CREATE INDEX idx_website_leads_interest_type ON website_leads(interest_type);
CREATE INDEX idx_catering_enquiries_status ON catering_enquiries(enquiry_status);
CREATE INDEX idx_delivery_enquiries_status ON delivery_enquiries(enquiry_status);
CREATE INDEX idx_customer_feedback_status ON customer_feedback(feedback_status);
```

9B. Update

backend/routes/api.php

Add these routes:

```
$router->get('/api/admin/website/leads', [WebsiteController::class, 'leads']);
$router->get('/api/admin/website/catering-enquiries', [WebsiteController::class, 'cateringEnquiries']);
$router->get('/api/admin/website/delivery-enquiries', [WebsiteController::class, 'deliveryEnquiries']);
$router->get('/api/admin/website/feedback', [WebsiteController::class, 'feedback']);
$router->get('/api/admin/website/acquisition-summary', [WebsiteController::class, 'acquisitionSummary']);

$router->post('/api/admin/website/leads/{id}/status', [WebsiteController::class, 'updateLeadStatus']);
$router->post('/api/admin/website/catering-enquiries/{id}/status', [WebsiteController::class,
'updateCateringStatus']);
$router->post('/api/admin/website/delivery-enquiries/{id}/status', [WebsiteController::class,
'updateDeliveryStatus']);
$router->post('/api/admin/website/feedback/{id}/status', [WebsiteController::class, 'updateFeedbackStatus']);
```

9C. Update

backend/app/Controllers/WebsiteController.php

Add these imports:

```
use Support\Auth;
```

Add these methods inside `WebsiteController`:

```
public function leads(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER']);

    $stmt = Database::connection()->query(
        'SELECT * FROM website_leads ORDER BY created_at DESC'
    );

    Response::success(['leads' => $stmt->fetchAll()]);
}

public function cateringEnquiries(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER']);

    $stmt = Database::connection()->query(
        'SELECT * FROM catering_enquiries ORDER BY created_at DESC'
    );

    Response::success(['catering_enquiries' => $stmt->fetchAll()]);
}
```

```
public function deliveryEnquiries(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER']);

    $stmt = Database::connection()->query(
        'SELECT * FROM delivery_enquiries ORDER BY created_at DESC'
    );

    Response::success(['delivery_enquiries' => $stmt->fetchAll()]);
}

public function feedback(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER']);

    $stmt = Database::connection()->query(
        'SELECT * FROM customer_feedback ORDER BY created_at DESC'
    );

    Response::success(['feedback' => $stmt->fetchAll()]);
}

public function acquisitionSummary(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER']);

    $db = Database::connection();

    $leads = $db->query('SELECT COUNT(*) AS total FROM website_leads')->fetch();
    $newLeads = $db->query("SELECT COUNT(*) AS total FROM website_leads WHERE lead_status = 'NEW'")->fetch();
    $converted = $db->query("SELECT COUNT(*) AS total FROM website_leads WHERE lead_status = 'CONVERTED'")-
>fetch();
    $catering = $db->query('SELECT COUNT(*) AS total FROM catering_enquiries')->fetch();
    $delivery = $db->query('SELECT COUNT(*) AS total FROM delivery_enquiries')->fetch();
    $feedback = $db->query('SELECT COUNT(*) AS total FROM customer_feedback')->fetch();

    Response::success([
        'total_leads' => (int) $leads['total'],
        'new_leads' => (int) $newLeads['total'],
        'converted_leads' => (int) $converted['total'],
        'catering_enquiries' => (int) $catering['total'],
        'delivery_enquiries' => (int) $delivery['total'],
        'feedback_count' => (int) $feedback['total'],
    ]);
}

public function updateLeadStatus(int $id): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        'UPDATE website_leads SET lead_status = :status WHERE id = :id'
    );

    $stmt->execute([
        'status' => $data['lead_status'],
        'id' => $id,
    ]);

    AuditLogger::log(
        (int) $user['id'],
        'WEBSITE',
        'UPDATE_LEAD_STATUS',
        $id,
        null,
        $data
    );

    Response::success([], 'Lead status updated');
}

public function updateCateringStatus(int $id): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        'UPDATE catering_enquiries SET enquiry_status = :status WHERE id = :id'
    );

    $stmt->execute([
        'status' => $data['enquiry_status'],
        'id' => $id,
    ]);

    AuditLogger::log(
```

```
        (int) $user['id'],
        'WEBSITE',
        'UPDATE_CATERING_STATUS',
        $id,
        null,
        $data
    );

    Response::success([], 'Catering status updated');
}

public function updateDeliveryStatus(int $id): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        'UPDATE delivery_enquiries SET enquiry_status = :status WHERE id = :id'
    );

    $stmt->execute([
        'status' => $data['enquiry_status'],
        'id' => $id,
    ]);

    AuditLogger::log(
        (int) $user['id'],
        'WEBSITE',
        'UPDATE_DELIVERY_STATUS',
        $id,
        null,
        $data
    );

    Response::success([], 'Delivery status updated');
}

public function updateFeedbackStatus(int $id): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        'UPDATE customer_feedback SET feedback_status = :status WHERE id = :id'
    );

    $stmt->execute([
        'status' => $data['feedback_status'],
        'id' => $id,
    ]);

    AuditLogger::log(
        (int) $user['id'],
        'WEBSITE',
        'UPDATE_FEEDBACK_STATUS',
        $id,
        null,
        $data
    );

    Response::success([], 'Feedback status updated');
}
```

9D.

apps/admin-web/src/features/website/api/websiteAdminApi.js

```
import apiClient from '../../../lib/apiClient'

export const websiteAdminApi = {
  summary: async () => {
    const { data } = await apiClient.get('/admin/website/acquisition-summary')
    return data.data
  },

  leads: async () => {
    const { data } = await apiClient.get('/admin/website/leads')
    return data.data.leads
  },

  catering: async () => {
    const { data } = await apiClient.get('/admin/website/catering-enquiries')
    return data.data.catering_enquiries
  },
```

```
  delivery: async () => {
    const { data } = await apiClient.get('/admin/website/delivery-enquiries')
    return data.data.delivery_enquiries
  },

  feedback: async () => {
    const { data } = await apiClient.get('/admin/website/feedback')
    return data.data.feedback
  },

  updateLeadStatus: async (id, lead_status) => {
    const { data } = await apiClient.post(`/admin/website/leads/${id}/status`, {
      lead_status,
    })
    return data
  },

  updateCateringStatus: async (id, enquiry_status) => {
    const { data } = await apiClient.post(`/admin/website/catering-enquiries/${id}/status`, {
      enquiry_status,
    })
    return data
  },

  updateDeliveryStatus: async (id, enquiry_status) => {
    const { data } = await apiClient.post(`/admin/website/delivery-enquiries/${id}/status`, {
      enquiry_status,
    })
    return data
  },

  updateFeedbackStatus: async (id, feedback_status) => {
    const { data } = await apiClient.post(`/admin/website/feedback/${id}/status`, {
      feedback_status,
    })
    return data
  },
}
```

9E.

apps/admin-web/src/features/website/pages/WebsiteAcquisitionDashboardPage.jsx

```
import React from 'react'
import { useQuery } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { websiteAdminApi } from '../api/websiteAdminApi'

export default function WebsiteAcquisitionDashboardPage() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['website-acquisition-summary'],
    queryFn: websiteAdminApi.summary,
  })

  return (
    <Page title="Website Acquisition" subtitle="AI-enabled customer acquisition performance">
      {isLoading ? <LoadingBlock label="Loading acquisition summary..." /> : null}
      {error ? <StatusMessage error={error.message} /> : null}

      {data ? (
        <div style={{ display: 'grid', gridTemplateColumns: 'repeat(3, minmax(0, 1fr))', gap: 16 }}>
          <Card title="Total Leads"><strong>{data.total_leads}</strong></Card>
          <Card title="New Leads"><strong>{data.new_leads}</strong></Card>
          <Card title="Converted Leads"><strong>{data.converted_leads}</strong></Card>
          <Card title="Catering Enquiries"><strong>{data.catering_enquiries}</strong></Card>
          <Card title="Delivery Enquiries"><strong>{data.delivery_enquiries}</strong></Card>
          <Card title="Feedback"><strong>{data.feedback_count}</strong></Card>
        </div>
      ) : null}
    </Page>
  )
}
```

9F.

apps/admin-web/src/features/website/pages/WebsiteLeadsPage.jsx

```
import React from 'react'
```

```
import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { websiteAdminApi } from '../api/websiteAdminApi'

export default function WebsiteLeadsPage() {
  const qc = useQueryClient()

  const { data, isLoading, error } = useQuery({
    queryKey: ['website-leads'],
    queryFn: websiteAdminApi.leads,
  })

  const mutation = useMutation({
    mutationFn: ({ id, status }) => websiteAdminApi.updateLeadStatus(id, status),
    onSuccess: () => qc.invalidateQueries({ queryKey: ['website-leads'] }),
  })

  const rows = (data || []).map((lead) => ({
    id: lead.id,
    full_name: lead.full_name,
    phone: lead.phone,
    interest_type: lead.interest_type,
    lead_status: (
      <select
        value={lead.lead_status}
        onChange={(e) => mutation.mutate({ id: lead.id, status: e.target.value })}
      >
        <option value="NEW">NEW</option>
        <option value="CONTACTED">CONTACTED</option>
        <option value="CONVERTED">CONVERTED</option>
        <option value="CLOSED">CLOSED</option>
      </select>
    ),
    created_at: lead.created_at,
  }))

  return (
    <Page title="Website Leads" subtitle="All customer leads captured from the website">
      {mutation.error ? <StatusMessage error={mutation.error.message} /> : null}
      <Card>
        {isLoading ? <LoadingBlock label="Loading leads..." /> : null}
        {error ? <StatusMessage error={error.message} /> : null}
        {!isLoading && !error ? (
          <DataTable
            columns={[
              { key: 'full_name', label: 'Name' },
              { key: 'phone', label: 'Phone' },
              { key: 'interest_type', label: 'Interest' },
              { key: 'lead_status', label: 'Status' },
              { key: 'created_at', label: 'Created' },
            ]}
            rows={rows}
          />
        ) : null}
      </Card>
    </Page>
  )
}
```

9G.

apps/admin-web/src/features/website/pages/CateringEnquiriesPage.jsx

```
import React from 'react'
import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { websiteAdminApi } from '../api/websiteAdminApi'

export default function CateringEnquiriesPage() {
  const qc = useQueryClient()

  const { data, isLoading, error } = useQuery({
    queryKey: ['catering-enquiries'],
    queryFn: websiteAdminApi.catering,
  })
```

```
  const mutation = useMutation({
    mutationFn: ({ id, status }) => websiteAdminApi.updateCateringStatus(id, status),
    onSuccess: () => qc.invalidateQueries({ queryKey: ['catering-enquiries'] }),
  })

  const rows = (data || []).map((item) => ({
    id: item.id,
    full_name: item.full_name,
    phone: item.phone,
    event_date: item.event_date,
    guest_count: item.guest_count,
    enquiry_status: (
      <select
        value={item.enquiry_status}
        onChange={(e) => mutation.mutate({ id: item.id, status: e.target.value })}
      >
        <option value="NEW">NEW</option>
        <option value="CONTACTED">CONTACTED</option>
        <option value="QUOTED">QUOTED</option>
        <option value="CONFIRMED">CONFIRMED</option>
        <option value="CLOSED">CLOSED</option>
      </select>
    ),
  }))

  return (
    <Page title="Catering Enquiries" subtitle="Website catering and events leads">
      <Card>
        {isLoading ? <LoadingBlock label="Loading catering enquiries..." /> : null}
        {error ? <StatusMessage error={error.message} /> : null}
        {!isLoading && !error ? (
          <DataTable
            columns={[
              { key: 'full_name', label: 'Name' },
              { key: 'phone', label: 'Phone' },
              { key: 'event_date', label: 'Event Date' },
              { key: 'guest_count', label: 'Guests' },
              { key: 'enquiry_status', label: 'Status' },
            ]}
            rows={rows}
          />
        ) : null}
      </Card>
    </Page>
  )
}
```

9H.

apps/admin-web/src/features/website/pages/DeliveryEnquiriesPage.jsx

```
import React from 'react'
import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { websiteAdminApi } from '../api/websiteAdminApi'

export default function DeliveryEnquiriesPage() {
  const qc = useQueryClient()

  const { data, isLoading, error } = useQuery({
    queryKey: ['delivery-enquiries'],
    queryFn: websiteAdminApi.delivery,
  })

  const mutation = useMutation({
    mutationFn: ({ id, status }) => websiteAdminApi.updateDeliveryStatus(id, status),
    onSuccess: () => qc.invalidateQueries({ queryKey: ['delivery-enquiries'] }),
  })

  const rows = (data || []).map((item) => ({
    id: item.id,
    full_name: item.full_name,
    phone: item.phone,
    delivery_address: item.delivery_address,
    order_request: item.order_request,
    enquiry_status: (
      <select
        value={item.enquiry_status}
        onChange={(e) => mutation.mutate({ id: item.id, status: e.target.value })}
      >
```

```
        <option value="NEW">NEW</option>
        <option value="CONTACTED">CONTACTED</option>
        <option value="CONFIRMED">CONFIRMED</option>
        <option value="DISPATCHED">DISPATCHED</option>
        <option value="CLOSED">CLOSED</option>
      </select>
    ),
  }))

  return (
    <Page title="Delivery Enquiries" subtitle="Website delivery demand and follow-up queue">
      <Card>
        {isLoading ? <LoadingBlock label="Loading delivery enquiries..." /> : null}
        {error ? <StatusMessage error={error.message} /> : null}
        {!isLoading && !error ? (
          <DataTable
            columns={[
              { key: 'full_name', label: 'Name' },
              { key: 'phone', label: 'Phone' },
              { key: 'delivery_address', label: 'Address' },
              { key: 'order_request', label: 'Request' },
              { key: 'enquiry_status', label: 'Status' },
            ]}
            rows={rows}
          />
        ) : null}
      </Card>
    </Page>
  )
}
```

91.

apps/admin-web/src/features/website/pages/FeedbackPage.jsx

```
import React from 'react'
import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { websiteAdminApi } from '../api/websiteAdminApi'

export default function FeedbackPage() {
  const qc = useQueryClient()

  const { data, isLoading, error } = useQuery({
    queryKey: ['customer-feedback'],
    queryFn: websiteAdminApi.feedback,
  })

  const mutation = useMutation({
    mutationFn: ({ id, status }) => websiteAdminApi.updateFeedbackStatus(id, status),
    onSuccess: () => qc.invalidateQueries({ queryKey: ['customer-feedback'] }),
  })

  const rows = (data || []).map((item) => ({
    id: item.id,
    full_name: item.full_name,
    phone: item.phone,
    rating: item.rating,
    message: item.message,
    feedback_status: (
      <select
        value={item.feedback_status}
        onChange={(e) => mutation.mutate({ id: item.id, status: e.target.value })}
      >
        <option value="NEW">NEW</option>
        <option value="REVIEWED">REVIEWED</option>
        <option value="ACTION_TAKEN">ACTION_TAKEN</option>
        <option value="CLOSED">CLOSED</option>
      </select>
    ),
  }))

  return (
    <Page title="Customer Feedback" subtitle="Website feedback and service improvement queue">
      <Card>
        {isLoading ? <LoadingBlock label="Loading feedback..." /> : null}
        {error ? <StatusMessage error={error.message} /> : null}
        {!isLoading && !error ? (
          <DataTable
            columns={[
```

```
              { key: 'full_name', label: 'Name' },
              { key: 'phone', label: 'Phone' },
              { key: 'rating', label: 'Rating' },
              { key: 'message', label: 'Message' },
              { key: 'feedback_status', label: 'Status' },
            ]}
            rows={rows}
          />
        ) : null}
      </Card>
    </Page>
  )
}
```

9J. Update

apps/admin-web/src/app/routes.jsx

Add imports:

```
import WebsiteAcquisitionDashboardPage from '../features/website/pages/WebsiteAcquisitionDashboardPage'
import WebsiteLeadsPage from '../features/website/pages/WebsiteLeadsPage'
import CateringEnquiriesPage from '../features/website/pages/CateringEnquiriesPage'
import DeliveryEnquiriesPage from '../features/website/pages/DeliveryEnquiriesPage'
import FeedbackPage from '../features/website/pages/FeedbackPage'
```

Add routes inside `DashboardLayout`:

```
<Route path="/website-acquisition" element={<WebsiteAcquisitionDashboardPage />} />
<Route path="/website-leads" element={<WebsiteLeadsPage />} />
<Route path="/catering-enquiries" element={<CateringEnquiriesPage />} />
<Route path="/delivery-enquiries" element={<DeliveryEnquiriesPage />} />
<Route path="/website-feedback" element={<FeedbackPage />} />
```

9K. Update

apps/admin-web/src/components/common/SidebarNav.jsx

Add these links:

```
['Website Acquisition', '/website-acquisition'],
['Website Leads', '/website-leads'],
['Catering Enquiries', '/catering-enquiries'],
['Delivery Enquiries', '/delivery-enquiries'],
['Website Feedback', '/website-feedback'],
```

9L. Update

apps/admin-web/src/features/dashboard/pages/DashboardPage.jsx

Replace file with:

```
import React from 'react'
import { Link } from 'react-router-dom'
import { useQuery } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { websiteAdminApi } from '../../website/api/websiteAdminApi'

export default function DashboardPage() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['dashboard-website-acquisition-summary'],
    queryFn: websiteAdminApi.summary,
  })

  return (
    <Page title="Dashboard" subtitle="Meat Lovers CIMS powered by YohPal">
      {isLoading ? <LoadingBlock label="Loading acquisition cards..." /> : null}
      {error ? <StatusMessage error={error.message} /> : null}

      {data ? (
        <div style={{ display: 'grid', gridTemplateColumns: 'repeat(3, minmax(0, 1fr))', gap: 16 }}>
          <Card title="Website Leads">
            <strong>{data.total_leads}</strong>
            <p><Link to="/website-leads">View leads</Link></p>
          </Card>

          <Card title="New Leads">
```

```
            <strong>{data.new_leads}</strong>
            <p><Link to="/website-acquisition">View acquisition</Link></p>
          </Card>

          <Card title="Converted Leads">
            <strong>{data.converted_leads}</strong>
            <p><Link to="/website-leads">Follow up</Link></p>
          </Card>

          <Card title="Catering">
            <strong>{data.catering_enquiries}</strong>
            <p><Link to="/catering-enquiries">View catering</Link></p>
          </Card>

          <Card title="Delivery">
            <strong>{data.delivery_enquiries}</strong>
            <p><Link to="/delivery-enquiries">View delivery</Link></p>
          </Card>

          <Card title="Feedback">
            <strong>{data.feedback_count}</strong>
            <p><Link to="/website-feedback">View feedback</Link></p>
          </Card>
        </div>
      ) : null}
    </Page>
  )
}
```

9M.

docs/ADMIN_WEBSITE_LEADS_MODULE.md

```
# Batch 9 — Admin Website Leads Module

## Purpose

This batch connects the AI-enabled website acquisition engine to the admin dashboard.

## Admin Can Now View

- website leads
- catering enquiries
- delivery enquiries
- customer feedback
- acquisition summary cards

## Admin Can Update Status

### Website Leads

- NEW
- CONTACTED
- CONVERTED
- CLOSED

### Catering Enquiries

- NEW
- CONTACTED
- QUOTED
- CONFIRMED
- CLOSED

### Delivery Enquiries

- NEW
- CONTACTED
- CONFIRMED
- DISPATCHED
- CLOSED

### Feedback

- NEW
- REVIEWED
- ACTION_TAKEN
- CLOSED

## Business Value

This makes the website measurable.

Meat Lovers management can now track:
```

```
- how many customers the website is bringing
- how many leads are new
- how many leads are converted
- how much catering demand exists
- how much delivery demand exists
- what customers are complaining about
- which leads need follow-up

## Enforcement

Website submissions are audit logged.

Status changes are audit logged.

The website is therefore not only public marketing; it becomes part of the CIMS control system.
```

Batch 9 Outcome

Admin now supports:

✅ leads listing page
✅ catering enquiries page
✅ delivery enquiries page
✅ feedback page
✅ lead status update API
✅ admin routes
✅ frontend API integration
✅ dashboard cards for acquisition performance

Next Smart Move

Build **Batch 10 — Admin Operational Modules UI**, including:

- suppliers page
- products page
- stock page
- bar page
- kitchen page
- dispatch page
- unsold food page
- assets page
- HRM page
- finance page
- income statement page